

DBS Project Part 1:

STUPIFY E-Commerce Database

Task 1a — Introduction & Initial Analysis

Introduction to Stupify

Stupify is a platform that serves as a digital storefront where customers can browse a wide array of products across various categories, add items to their shopping carts, place orders, make secure payments, and leave reviews based on their experiences.

As Stupify's user base and product inventory expand, relying on manual data management or simple spreadsheets becomes unsustainable and error-prone. To ensure smooth operations, Stupify requires a robust, centralized system to manage its data efficiently.

Purpose of the Database

The proposed database is a Relational Database designed to store, manage, and retrieve all critical business information—such as customer details, product catalogs, order histories, and payment records—in one secure and organized place.

In a relational database, data is stored in tables that are linked together via primary and foreign keys to prevent duplication and ensure accuracy.

Daily Business Transactions Supported

The database will execute the following core transactions to support Stupify's daily operations:

1. **Customer Management:** Registering new customers, updating contact details, and managing multiple delivery addresses.
2. **Product Management:** Adding new products, updating pricing, and dynamically adjusting stock levels as purchases occur.
3. **Cart Operations:** Allowing customers to add, modify, or remove items in their shopping carts before checkout.
4. **Order Processing:** Creating new orders, calculating total amounts, and tracking order statuses from 'Pending' to 'Delivered'.
5. **Payment Processing:** Recording payment transactions and securely tracking payment statuses (e.g., Completed, Refunded).
6. **Shipping:** Managing shipping methods (e.g., Standard, Express) and applying appropriate shipping costs.
7. **Reviews:** Storing customer ratings and textual feedback for products to help other buyers.

Entities

To model Stupify's business accurately, the following 12 entities (tables) will be implemented:

1. **CUSTOMER**
2. **ADDRESS**
3. **POSTAL_CODE**
4. **CART**
5. **CART_ITEM**
6. **ORDER**
7. **ORDER_ITEM**
8. **PRODUCT**
9. **CATEGORY**
10. **PAYMENT**
11. **SHIPPING_METHOD**
12. **REVIEW**

Advantages and Disadvantages of a Relational Database for Stupify

Advantages:

- **Data Integrity and Accuracy:** By using Foreign Keys, Stupify ensures that orders cannot be placed for non-existent products or by unregistered customers.
- **Reduced Redundancy:** Customer details are stored once, rather than being repeated on every single order, saving space and preventing conflicting information.
- **Powerful Reporting:** Stupify can easily run complex operations/queries to find products, calculate monthly revenue, or identify the most active customers.

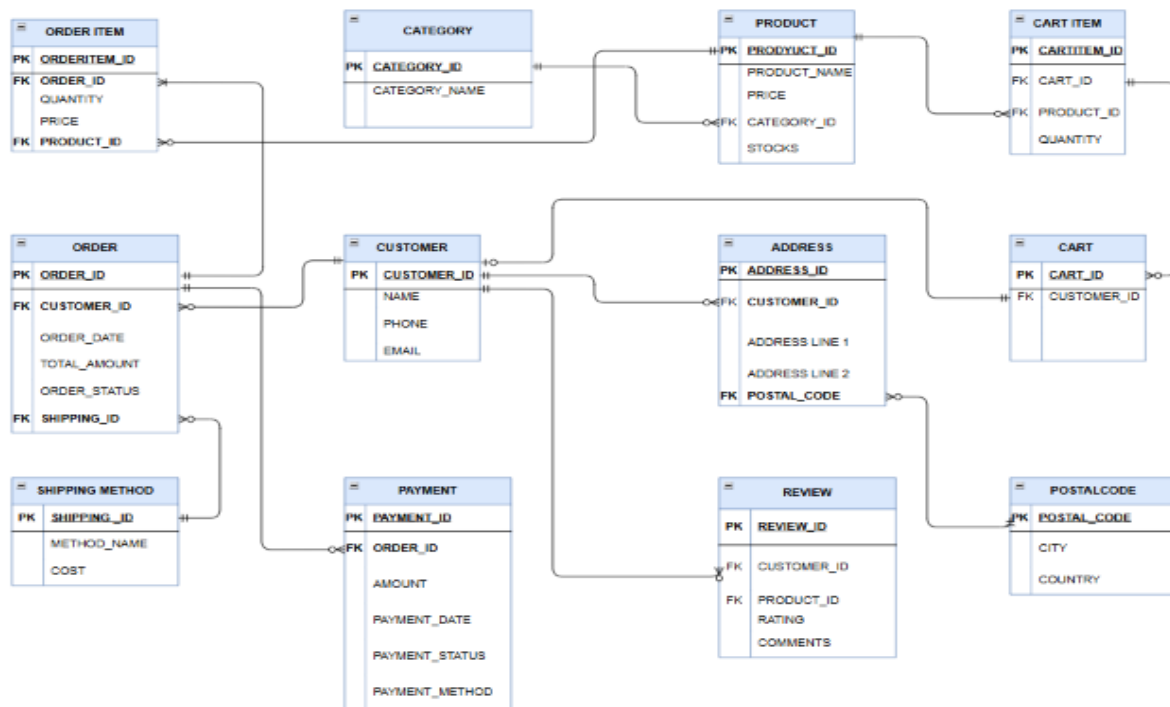
Disadvantages:

- **Setup Complexity:** Designing a normalized schema requires significant upfront planning and technical expertise.
- **Maintenance Overhead:** The system requires ongoing maintenance, backups, and specialized technical knowledge to keep it running.

- **Scaling Limits:** While relational databases handle structured data perfectly, scaling them across multiple servers (horizontal scaling) as Stupify grows into a massive enterprise can be difficult and expensive.

Task 1b — ERD & Normalization to 3NF

Entity Relationship Diagram (ERD) Overview



Key Relationships, Cardinality, and Optionality:

- **Customer to Order (1:M):** A customer can place zero or many orders, but each order must belong to exactly one customer.
- **Customer to Address (1:M):** A customer may have zero or many addresses, but each address must belong to exactly one customer.
- **Postal Code to Address (1:M):** A postal code can apply to zero or many addresses, but each address must have exactly one postal code.
- **Customer to Cart (1:1):** A customer has at most one active cart, and each cart belongs to exactly one customer.
- **Cart to Cart Item (1:M):** A cart may contain zero or many items, but each cart item must belong to exactly one cart.

- **Order to Order Item (1:M):** An order must contain one or many order items, and each order item must belong to exactly one order.
- **Product to Order Item (1:M):** A product can be part of zero or many order items, but each order item must reference exactly one product.
- **Category to Product (1:M):** A category can contain zero or many products, but each product must belong to exactly one category.
- **Order to Payment (1:M):** An order can have zero or many payment attempts (e.g., failed then completed), but each payment record must link to exactly one order.
- **Shipping Method to Order (1:M):** A shipping method can be used for zero or many orders, but an order must select exactly one shipping method.
- **Product to Review (1:M) & Customer to Review (1:M):** A customer can write zero or many reviews, and a product can receive zero or many reviews. Each review must link to exactly one customer and exactly one product.

Normalization Process

Unnormalized Form (UNF): All data is conceptually stored in a single, massive table containing repeating groups (e.g., an order record containing multiple products, prices, and quantities directly within the row).

First Normal Form (1NF): Rule: Remove repeating groups; ensure all attributes are atomic. We separate the repeating groups into their own tables. For example, instead of having a list of products inside an ORDER, we create an ORDER_ITEM table.

- **CUSTOMER (Customer_ID, Name, Email, Phone)**
- **ORDER (Order_ID, Customer_ID, Date, Total_Amount)**
- **ORDER_ITEM (Order_ID, Product_ID, Quantity, Price)**

Second Normal Form (2NF): Rule: Must be in 1NF; remove partial dependencies (attributes must depend on the whole primary key). In tables with composite keys like ORDER_ITEM (Order_ID, Product_ID), if we had Product_Name stored there, it would only depend on Product_ID. We remove it by ensuring Product is its own separate table.

- **PRODUCT (Product_ID, Product_Name, Price, Category_ID, Stocks)**

Third Normal Form (3NF): Rule: Must be in 2NF; remove transitive dependencies (non-key attributes depending on other non-key attributes). If the ADDRESS table contained City and Country alongside Postal_Code, City and Country would transitively depend on Postal_Code rather than Address_ID. We resolve this by creating a separate POSTAL_CODE table.

- **ADDRESS** (Address_ID, Customer_ID, Address_Line1, Postal_Code)
- **POSTAL_CODE** (Postal_Code, City, Country)

Problems Encountered and Resolved

- **Problem:** M:M relationships between Orders and Products caused data redundancy and made querying difficult.
- **Resolution:** Introduced an associative (bridge) entity called ORDER_ITEM to resolve the M:M relationship into two 1:M relationships, allowing us to store historical prices and exact quantities per order.
- **Problem:** Normalization conflicts regarding address storage. Customers often have multiple addresses (home, work), which create repeating columns or rows in the Customer table.
- **Resolution:** Separated addresses into an independent ADDRESS table linked to CUSTOMER via a Foreign Key, satisfying 1NF. Further normalized location data into a POSTAL_CODE table to satisfy 3NF.

Task 1c — Data Dictionary

Table Name	Attribute Name	Key	Data Type	Constraints	Description
CUSTOMER	customer_id	PK	INT	AUTO_INCREMENT	Unique ID for the customer
	customer_name		VARCHAR(50)	NOT NULL	Full name of the customer
	phone		VARCHAR(11)		Customer's phone number
	email		VARCHAR(320)	UNIQUE	Standard max length email address
POSTAL_CODE	postal_code	PK	VARCHAR(20)		Zip/Postal code string
	city		VARCHAR(50)		City name

	country		VARCHAR(50)		Country name
ADDRESS	address_id	PK	INT	AUTO_INCREMENT	Unique ID for address record
	customer_id	FK	INT		Links to CUSTOMER
	address_line1		VARCHAR(100)		Primary address text
	address_line2		VARCHAR(100)		Secondary address text
	postal_code	FK	VARCHAR(20)		Links to POSTAL_CODE
CATEGORY	category_id	PK	INT	AUTO_INCREMENT	Unique ID for product category
	category_name		VARCHAR(100)		Name of the category
PRODUCT	product_id	PK	INT	AUTO_INCREMENT	Unique ID for product
	product_name		VARCHAR(100)		Title of the product
	price		DECIMAL(15,2)		Current listing price
	category_id	FK	INT		Links to CATEGORY
	stocks		INT		Current inventory count
CART	cart_id	PK	INT	AUTO_INCREMENT	Unique ID for a cart
	customer_id	FK	INT		Links to CUSTOMER

CART_ITEM	cartitem_id	PK	INT	AUTO_INCREMENT	Unique ID for cart line item
	cart_id	FK	INT		Links to CART
	product_id	FK	INT		Links to PRODUCT
	quantity		INT		Number of units added to cart
SHIPPING_METHOD	shipping_id	PK	INT	AUTO_INCREMENT	Unique ID for shipping method
	method_name		ENUM	NOT NULL	'Standard', 'Express', 'Overnight', 'Free Shipping'
	shipping_cost		DECIMAL(15,2)		Cost applied to order
ORDER	order_id	PK	INT	AUTO_INCREMENT	Unique ID for the order
	customer_id	FK	INT		Links to CUSTOMER
	order_date		DATE		Date the order was placed
	total_amount		DECIMAL(15,2)		Total bill for the order
	order_status		ENUM		'Pending', 'Processing', 'Shipped', 'Delivered', 'Cancelled'
	shipping_id	FK	INT		Links to SHIPPING_METHOD

ORDER_ITEM	orderid_id	PK	INT	AUTO_INCREMENT	Unique ID for order line item
	order_id	FK	INT		Links to ORDER
	product_id	FK	INT		Links to PRODUCT
	quantity		INT		Number of units purchased
	price		DECIMAL(15,2)		Historical price at time of purchase
PAYMENT	payment_id	PK	INT	AUTO_INCREMENT	Unique ID for payment record
	order_id	FK	INT		Links to ORDER
	amount		DECIMAL(15,2)		Amount paid
	payment_date		DATE		Date of payment
	payment_status		ENUM		'Pending', 'Completed', 'Failed', 'Refunded'
	payment_method		ENUM		'Credit Card', 'Debit Card', 'PayPal', 'Cash on Delivery'
REVIEW	review_id	PK	INT	AUTO_INCREMENT	Unique ID for review
	customer_id	FK	INT		Links to CUSTOMER
	product_id	FK	INT		Links to PRODUCT

	rating		INT	CHECK (1-5)	Numeric rating out of 5
	comments		TEXT		Textual review

Task 2b — Sample Data

The database has been populated with 1,109 rows of realistic sample data generated using Claude.ai. The data accurately reflects a Pakistani e-commerce store's data.

Task 2c — SQL Queries

1:

```
USE stupify;

WITH product_sales AS (
    SELECT
        p.product_id,
        p.product_name,
        SUM(oi.quantity) AS total_qty_sold,
        COUNT(oi.orderitem_id) AS times_ordered
    FROM product p
    JOIN order_item oi ON p.product_id = oi.product_id
    GROUP BY p.product_id, p.product_name
)
SELECT
    product_name,
    total_qty_sold,
    times_ordered
FROM product_sales
ORDER BY total_qty_sold ASC
LIMIT 5;
```

Output:

Result(RO) X			
Q Search Results			
	product name varchar	total_qty_sold decimal	times_ordered bigint
>	Interwood Study Table with	1	1
>	iPhone 15 Pro Max	1	1
>	OPPO A78	1	1
>	Khaadi Unstitched Fabric 2p	1	1
>	Barbie Dreamhouse Playset	1	1

2:

```
WITH category_revenue AS (  
    SELECT  
        c.category_name,  
        COUNT(DISTINCT o.order_id) AS total_orders,  
        SUM(oi.quantity * oi.price) AS total_revenue  
    FROM category c  
    JOIN product p ON c.category_id = p.category_id  
    JOIN order_item oi ON p.product_id = oi.product_id  
    JOIN `order` o ON oi.order_id = o.order_id  
    WHERE o.order_status = 'Delivered'  
    AND oi.price > 0  
    GROUP BY c.category_name  
)  
SELECT  
    category_name,  
    total_orders,  
    total_revenue  
FROM category_revenue  
ORDER BY total_revenue ASC;
```

Output:

Result(RO) X		
Search Results		
Cost: 47ms < 1 > Total 9		
category_name	total_orders	total_revenue
varchar	bigint	decimal
> Toys & Games	1	998.00
> Beauty & Personal Care	3	14392.00
> Jewellery & Watches	1	31998.00
> Clothing & Apparel	11	108966.00
> Footwear	19	453960.00
> Furniture	4	485992.00
> Home & Kitchen	12	498964.00
> Mobile Phones & Accessories	28	3766916.00
> Electronics	23	4949954.00

3:

```
USE stupify;

WITH completed_payers AS (
    SELECT DISTINCT order_id
    FROM payment
    WHERE payment_status = 'Completed'
),
all_orders AS (
    SELECT DISTINCT customer_id, order_id
    FROM `order`
    WHERE order_status != 'Cancelled'
)
SELECT
    c.customer_id,
    c.customer_name,
    c.email
FROM customer c
JOIN all_orders ao ON c.customer_id = ao.customer_id
WHERE ao.order_id NOT IN (
    SELECT order_id FROM completed_payers
)
AND c.email IS NOT NULL
ORDER BY c.customer_name ASC;
```

Output:

Result(RO) X		
Q Search Results		
Cost: 38ms < 1 2 > Total 113		
customer_id int	customer_name varchar	email varchar
> 33	Adil Hussain	adil.hussain@yahoo.com
> 33	Adil Hussain	adil.hussain@yahoo.com
> 75	Affan Waheed	affan.waheed@hotmail.com
> 71	Ahad Raza	ahad.raza@yahoo.com
> 51	Ahsan Iqbal	ahsan.iqbal@gmail.com
> 51	Ahsan Iqbal	ahsan.iqbal@gmail.com
> 26	Aisha Farooq	aisha.farooq@gmail.com
> 1	Ali Hassan	ali.hassan@gmail.com
> 1	Ali Hassan	ali.hassan@gmail.com
> 20	Amna Baig	amna.baig@gmail.com
> 20	Amna Baig	amna.baig@gmail.com
> 56	Areeba Habib	areeba.habib@gmail.com

4:

```
SELECT
    p.product_name,
    COUNT(r.review_id)      AS total_reviews,
    AVG(r.rating)           AS avg_rating,
    MAX(r.rating)           AS highest_rating,
    MIN(r.rating)           AS lowest_rating
FROM product p
JOIN review r ON p.product_id = r.product_id
WHERE r.rating IS NOT NULL
AND r.rating BETWEEN 1 AND 5
GROUP BY p.product_id, p.product_name
HAVING COUNT(r.review_id) >= 2
ORDER BY avg_rating ASC;
```

Output:

product_name	total_reviews	avg_rating	highest_rating	lowest_rating
Tapal Danedar 900g Tea	2	3.0000	3	3
Alkaram Winter Shawl	2	3.0000	3	3
Dove Deeply Nourishing Bo	2	3.0000	3	3
Alif by Umera Ahmed (Nove	2	3.0000	3	3
Borjan Casual Sneakers	2	3.0000	3	3
Reebok Gym Gloves	4	3.0000	3	3
Adidas Ultraboost 22	12	3.3333	4	3
Tecno Camon 20	8	3.5000	4	3
Khaadi Unstitched Fabric 2p	2	4.0000	4	4
Anker PowerCore 20000mA	2	4.0000	4	4

5:

```
UPDATE `order`
SET    order_status = 'Processing'
WHERE  order_status = 'Pending'
AND    order_date   < '2024-06-01';
SELECT *
```

Output:

order_id	customer_id	order_date	total_amount	order_status	shipping_id
1	1	2024-01-05	204998.00	Delivered	4
2	2	2024-01-07	399999.00	Delivered	2
3	3	2024-01-10	62498.00	Delivered	1
4	4	2024-01-12	54999.00	Delivered	1
5	5	2024-01-15	149999.00	Delivered	3
6	6	2024-01-18	5998.00	Delivered	4
7	7	2024-01-20	14999.00	Delivered	1
8	8	2024-01-22	4999.00	Delivered	4
9	9	2024-01-25	109999.00	Delivered	2
10	10	2024-01-28	29999.00	Delivered	1

6:

```
DELETE FROM review
WHERE (comments IS NULL OR comments = '')
AND rating < 3
SELECT *
FROM review;
```

Output:

review_id	customer_id	product_id	rating	comments
1	1	1	5	Samsung TV ki picture quali
2	1	22	4	Anker power bank bohat rel
3	2	12	5	iPhone 15 Pro Max - paise
4	3	13	4	Redmi Note 13 is great valu
5	3	51	3	Python book is okay, could
6	4	14	4	OPPO A78 is a solid mid-rar
7	5	3	5	Dell laptop bohot fast hai. C
8	6	26	4	Khaadi fabric ki quality umd
9	6	57	5	Keune shampoo use karne k
10	7	30	3	Alkaram shawl achi quality k

Task 2d — Transactions

The Importance of Atomicity in E-Commerce (Business Justification): Atomicity guarantees that a transaction is treated as a single, indivisible "all-or-nothing" operation. In Stupify's e-commerce operations, cancelling an order requires three distinct database changes: updating the order status, issuing a refund in the payment table, and restoring the product inventory stock. If atomicity is not enforced and the server crashes after the refund is processed but before the stock is restored, Stupify would lose money on the refund and lose the un-restored inventory item, causing severe data corruption and financial loss.

```
USE stupify;
SELECT o.order_id, o.customer_id, o.order_status, oi.product_id, p.stocks, pay.payment_status
FROM `order` o
JOIN order_item oi ON o.order_id = oi.order_id
JOIN product p ON oi.product_id = p.product_id /*a general overview of the product and their details
JOIN payment pay ON o.order_id = pay.order_id
WHERE o.order_status = 'Delivered'
LIMIT 10;
```

Result(RO) X						
Search Results						
order_id	customer_id	order_status	product_id	stocks	payment_status	
int	int	string	int	int	string	
> 1	1	Delivered	1	23	Completed	
> 1	1	Delivered	22	80	Completed	
> 3	3	Delivered	13	79	Completed	
> 3	3	Delivered	23	300	Completed	
> 4	4	Delivered	14	65	Completed	
> 5	5	Delivered	3	15	Completed	
> 6	6	Delivered	26	100	Completed	
> 6	6	Delivered	57	150	Completed	

```

SELECT o.order_id, o.customer_id, o.order_status, oi.product_id, p.stocks, pay.payment_status
FROM `order` o
JOIN order_item oi ON o.order_id = oi.order_id
JOIN product p ON oi.product_id = p.product_id /* The SELECT statement before the cancellation*/
JOIN payment pay ON o.order_id = pay.order_id
WHERE o.order_id = 5;

/*CANCELLATION*/

```

Result(RO) X						
Search Results						
order_id	customer_id	order_status	product_id	stocks	payment_status	
int	int	string	int	int	string	
> 5	5	Delivered	3	15	Completed	

Cancellation of product:

```

16
17 START TRANSACTION;
18 UPDATE `order`
19 SET order_status='Cancelled'
20 WHERE order_id=5;
21 UPDATE payment
22 SET payment_status = 'Refunded'
23 WHERE order_id=5;
24 UPDATE product
25 SET stocks=stocks+1
26 WHERE product_id IN (SELECT product_id
27 FROM order_item
28 WHERE order_id=5);
29 COMMIT;

```

After Cancellation:

```

30
31 SELECT o.order_id, o.order_status, pay.payment_status, p.product_id, p.stocks
32 FROM `order` o
33 JOIN payment pay ON o.order_id = pay.order_id
34 JOIN order_item oi ON o.order_id = oi.order_id
35 JOIN product p ON oi.product_id = p.product_id
36 WHERE o.order_id = 5;

```

Result(RO) X					
Search Results					
order_id	order_status	payment_status	product_id	stocks	
int	string	string	int	int	
> 5	Cancelled	Refunded	3	17	

ROLLBACK:

Before ROLLBACK:

```

39
40 SELECT o.order_id, o.customer_id, o.order_status, oi.product_id, p.stocks, pay.payment_status
41 FROM `order` o
42 JOIN order_item oi ON o.order_id = oi.order_id
43 JOIN product p ON oi.product_id = p.product_id
44 JOIN payment pay ON o.order_id = pay.order_id
45 WHERE o.order_id = 4;
46

```

```

START TRANSACTION;
UPDATE `order`
SET order_status='Cancelled'
WHERE order_id=4;
UPDATE payment
SET payment_status = 'Refunded'
WHERE order_id=4;
UPDATE product
SET stocks=stocks+1
WHERE product_id IN (SELECT product_id
FROM order_item
WHERE order_id=4);
ROLLBACK;

```

After ROLLBACK:

Result(RO) X					
Search Results					
order_id	customer_id	order_status	product_id	stocks	payment_status
int	int	string	int	int	string
> 4	4	Delivered	14	65	Completed

Task 2e — Role-Based Access Control (RBAC)

Role 1: Admin (admin_2)

- **Justification:** The admin role has full privileges because it represents the system owner or database administrator responsible for overall control of the platform. This role requires unrestricted access to manage all tables, maintain system integrity, and handle schema changes. Full access ensures the administrator can fix issues, update structure, and oversee all operations efficiently.

- **Privileges: ALL PRIVILEGES**

Role 2: Database Manager (database_manager)

Justification: The database manager role is given limited write access to products and read access to key business tables because they are responsible for maintaining daily business data. They need to update product information, manage stock, and monitor orders, but should not be allowed to delete or modify database structure. This restriction prevents accidental or intentional damage to critical data while still allowing operational control.

- **Privileges: SELECT, INSERT, UPDATE on PRODUCT; SELECT on CATEGORY, CUSTOMER, ORDER.**

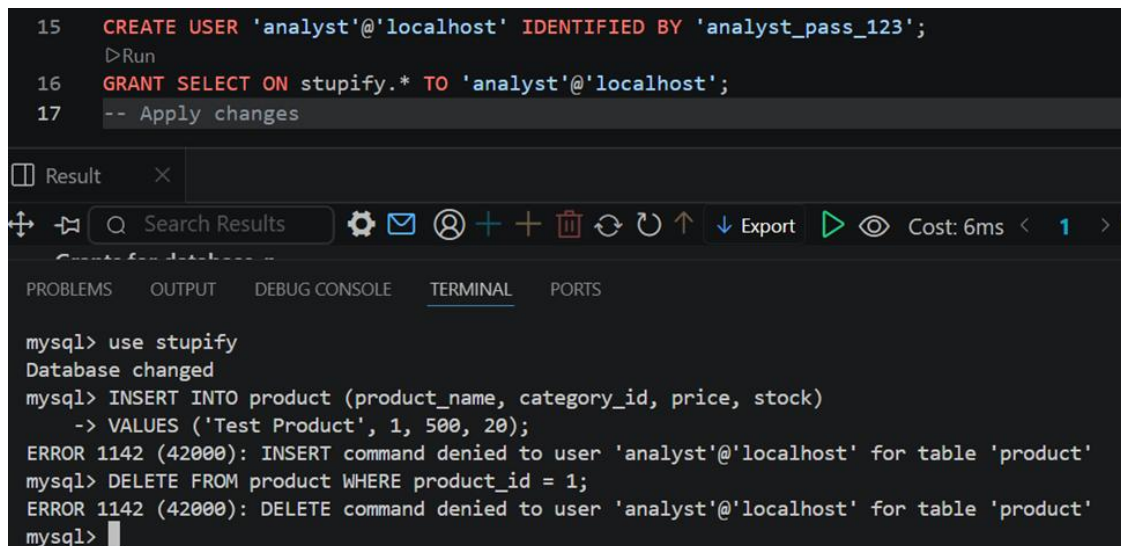
Role 3: Data Analyst (analyst)

- **Justification:** The analyst role is given read-only access because their responsibility is to analyze data and generate insights rather than modify it. They need access to view customers, products, and sales data to produce reports and trends. Restricting write operations ensures data integrity is maintained while still allowing effective business analysis.
- **Privileges: SELECT on all tables.**

SQL Implementation

```
1  use stupify;
2
3  -- 1. ADMIN ROLE - Full access to everything
4  ▷Run
5  CREATE USER 'admin_2'@'localhost' IDENTIFIED BY 'admin_pass123';
6  ▷Run
7  GRANT ALL PRIVILEGES ON stupify.* TO 'admin_2'@'localhost';
8  -- 2. manager ROLE - Can view and add products, but not delete
9  ▷Run
10 CREATE USER 'database_manager'@'localhost' IDENTIFIED BY 'manager_pass_123';
11 ▷Run
12 GRANT SELECT, INSERT, UPDATE ON stupify.product TO 'database_manager'@'localhost';
13 ▷Run
14 GRANT SELECT ON stupify.category TO 'database_manager'@'localhost';
15 ▷Run
16 GRANT SELECT ON stupify.customer TO 'database_manager'@'localhost';
17 ▷Run
18 GRANT SELECT ON stupify.`order` TO 'database_manager'@'localhost';
19 ▷Run
20 SHOW GRANTS FOR 'database_manager'@'localhost';
21
22 -- 3. ANALYST ROLE - Read-only access
23 ▷Run
24 CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'analyst_pass_123';
25 ▷Run
26 GRANT SELECT ON stupify.* TO 'analyst'@'localhost';
27 -- Apply changes
28 ▷Run
29 FLUSH PRIVILEGES;
```

Denial of Access:



The screenshot shows a MySQL command-line interface with a dark theme. At the top, a code editor displays three lines of SQL: `15 CREATE USER 'analyst'@'localhost' IDENTIFIED BY 'analyst_pass_123';`, `16 GRANT SELECT ON stupify.* TO 'analyst'@'localhost';`, and `17 -- Apply changes`. Below the editor is a toolbar with icons for search, settings, and other functions. The main terminal area shows the following output: `mysql> use stupify` followed by `Database changed`. Then, `mysql> INSERT INTO product (product_name, category_id, price, stock)` followed by `-> VALUES ('Test Product', 1, 500, 20);` results in `ERROR 1142 (42000): INSERT command denied to user 'analyst'@'localhost' for table 'product'`. Finally, `mysql> DELETE FROM product WHERE product_id = 1;` results in `ERROR 1142 (42000): DELETE command denied to user 'analyst'@'localhost' for table 'product'`. The prompt `mysql>` is visible at the bottom.

Task 3 — Web Application Front End

(Note: As per instructions, the Web Application files (HTML/CSS/Python) will be submitted as a separate directory. Below is the documentation component for the web app).

Web App Documentation

Frontend-Database Connection

The Stupify frontend connects with the MySQL database through a Flask backend. When a user interacts with the web interface—clicking a table name, searching records, or editing data—the browser sends HTTP requests (GET/POST) to Flask routes. Flask then executes SQL queries using the `mysql.connector` library, retrieves data from the database, and renders it back to the frontend as HTML templates (Jinja2). For dynamic operations like editing and deleting records, JavaScript handles client-side form submission and sends JSON responses back to update the UI with alerts rather than page reloads, creating a seamless user experience.

Limitations Encountered:

1. Table Name Case Sensitivity – MySQL table names are lowercase, but the backend was converted to uppercase, causing table switching to fail. Fixed by normalizing to lowercase.
2. Dynamic Column Display – The system displays all columns from any table, but some tables have many fields, making the interface crowded. A way to select the column to be displayed would make the UI better overall.
3. Search Limitations – Search only works on the second column of each table. A full-text search across all columns would result in better functionality.

4. Update Permissions – Managers can only update specific columns on PRODUCT. Cross-table updates are restricted by MySQL user roles, limiting flexibility for complex operations when it comes to access control.
5. Order of log ins- was difficult to determine when the user was to be asked to log in, either before the home page or before the records or add product page.

Stupify DB Home Records Add Product Read MeMANAGER — DATABASE_MANAGERMenu

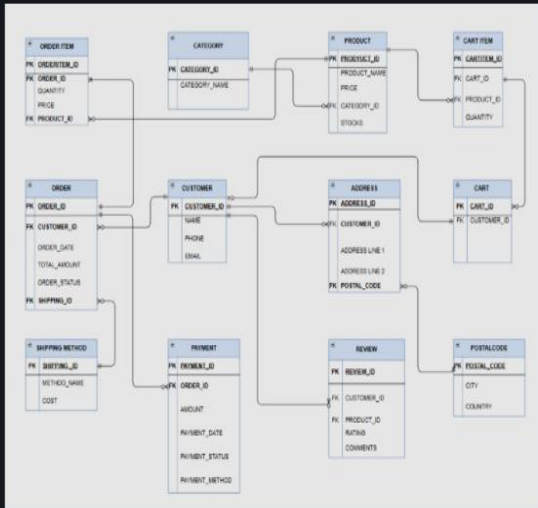
LIVE DATABASE INTERFACE

Access your store with clarity.

A database-powered interface for your Shopify store.
Browse records, run searches, add products, and manage data — all from one clean dashboard.

View Records →Learn More

Stupify DB Home Records Add Product Read MeMANAGER — DATABASE_MANAGERMenu



ABOUT THIS PROJECT

A database built for a Shopify store.

Stupify is a database management interface designed for a simplified Shopify-style e-commerce platform. It connects a structured MySQL backend to a clean HTML/CSS front end via a Flask server.

The schema covers everything from customers and orders to payments, shipping, reviews, and product categories — all visible in the ERD to the left.

HTML & CSSPython / FlaskMySQLJinja2

Stupify DB

HomeRecordsAdd ProductRead Me

MANAGER — DATABASE_MANAGER

Tables

product

View, search, and manage records from your database tables.

Search records...

Search

PRODUCT_ID	PRODUCT_NAME	PRICE	CATEGORY_ID	STOCKS	TEST
1	7	189999.00	1	25	None
2	LG 43" Full HD LED TV	89999.00	1	40	None
3	Dell Inspiron 15 Laptop	149999.00	1	16	None

address

cart

cart_item

category

customer

order

order_item

payment

postal_code

product

review

shipping_method

Stupify DB

HomeRecordsAdd ProductAbout

MANAGER — DATABASE_MANAGER

Menu

CONTENTS

About

Team

Database Schema

Features

Tech Stack

About Stupify

Stupify is a database management interface designed for e-commerce store operations. Built as a simplified Shopify-style database system, Stupify enables store managers and staff to efficiently manage products, customers, orders, and inventory through an intuitive web interface connected to a robust MySQL relational database.

The platform demonstrates enterprise-grade database design with role-based access control, allowing different user types (Admin, Staff, Analyst) to interact with data according to their responsibilities. All transactions are managed through a Python Flask backend that connects to a normalized MySQL database adhering to third normal form (3NF) principles.

Team

Stupify DB

HomeRecordsAdd ProductRead Me

MANAGER — DATABASE_MANAGERMenu

PRODUCT NAME

e.g. Classic White T-Shirt

PRICE (USD)STOCK QUANTITY

0.000

CATEGORY ID

e.g. 3

Add Product →Cancel

Stupify DB	Home	Records	Add Product	About	MANAGER — DATABASE_MANAGER	Menu
CONTENTS	About	Team	Database Schema	Features	Tech Stack	
				Features • Role-Based Access Control — Three distinct user roles (Admin, Staff, Analyst) with granular MySQL-enforced privileges • Product Management — View, search, filter, and add products with real-time inventory tracking • Normalized Database Design — All tables conform to Third Normal Form (3NF) eliminating data redundancy • Transaction Support — ACID-compliant transactions ensuring data consistency across operations • Clean Web Interface — Dark-themed, responsive design built with HTML, CSS, and Flask • Real-Time Queries — Live data retrieval from MySQL with JOINS and aggregations		